

Read and update data with Room

1. Before you begin
2. Starter app overview
3. Add a RecyclerView
4. Display item details
5. Implement sell item
6. Solution code
7. Summary
8. Learn more

[1. Before you begin](#)

You have learned in the previous codelabs how to use a Room persistence library, an abstraction layer on top of a [SQLite](#) database to store the app data. In this codelab, you'll add more features to the Inventory app and learn how to read, display, update, and delete data from the SQLite database using Room. You will use a `RecyclerView` to display the data from the database and automatically update the data when the underlying data in the database is changed.

Prerequisites

- You know how to create and interact with the SQLite database using the Room library.
- You know how to create an entity, DAO, and database classes.
- You know how to use a data access object (DAO) to map Kotlin functions to SQL queries.
- You know how to display list items in a `RecyclerView`.
- You've taken the previous codelab in this unit, [Persisting data with Room](#)

What you'll learn

- How to read and display entities from a SQLite database.
- How to update and delete entities from a SQLite database using the Room library.

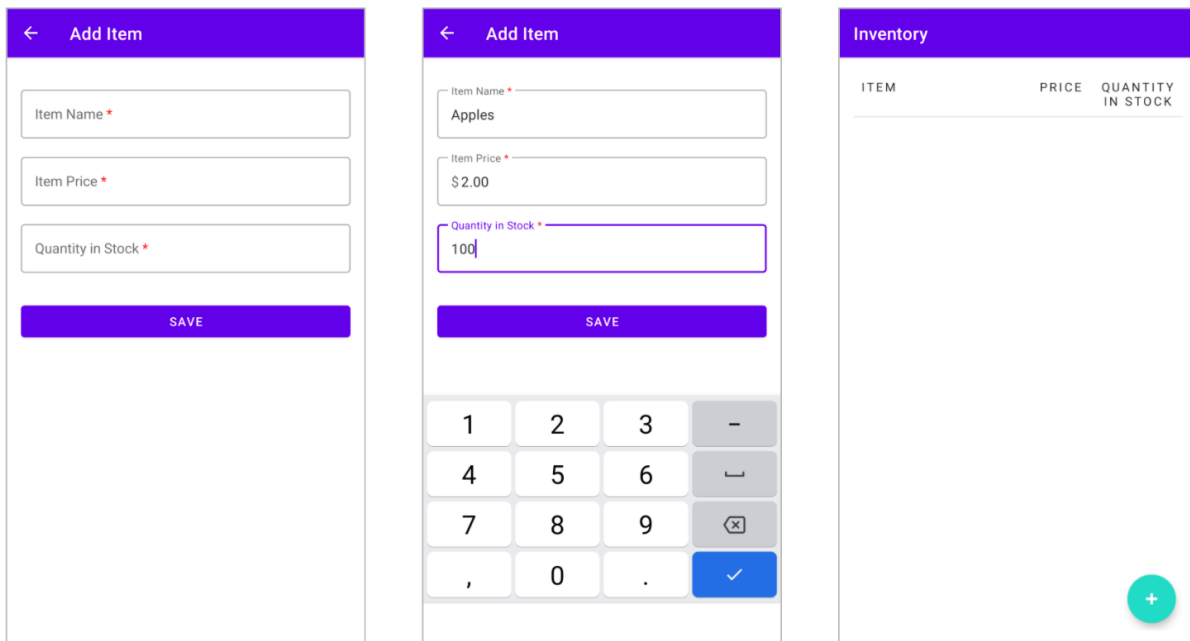
What you'll build

- You'll build an Inventory app that displays a list of inventory items. The app can update, edit, and delete items from the app database using Room.

2. Starter app overview

This codelab uses the Inventory app solution code from the [previous codelab](#) as the starter code. The starter app already saves data using the [Room](#) persistence library. The user can add data to the app database using the **Add Item** screen.

Note: The current version of the starter app doesn't display the data stored in the database.



In this codelab, you will extend the app to read and display the data, update and delete entities on the database using Room library.

Download the starter code for this codelab

This starter code is the same as the solution code from the previous codelab.

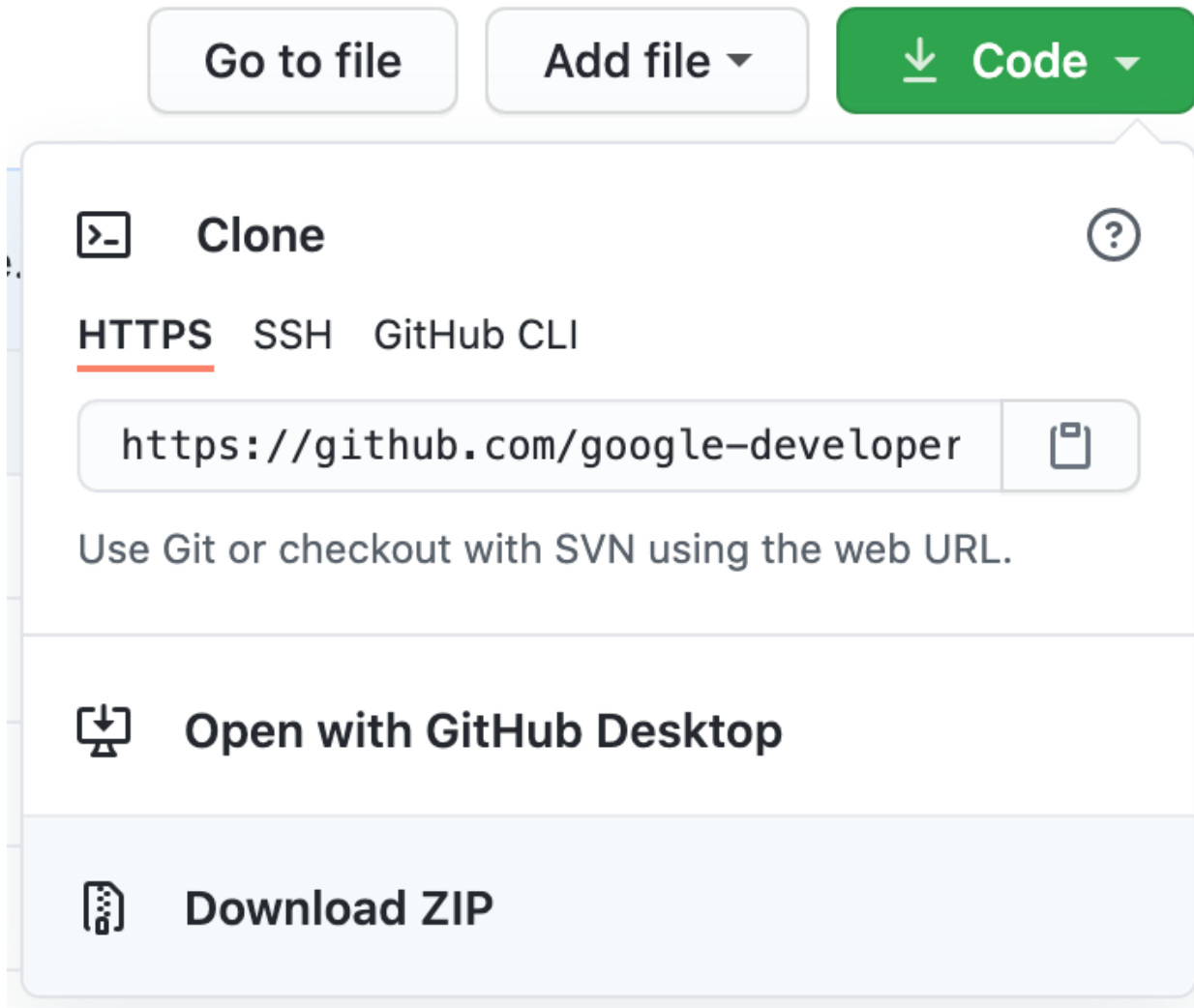
Starter Code URL: <https://github.com/google-developer-training/android-basics-kotlin-inventory-app/tree/room>

Branch name: `room`

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

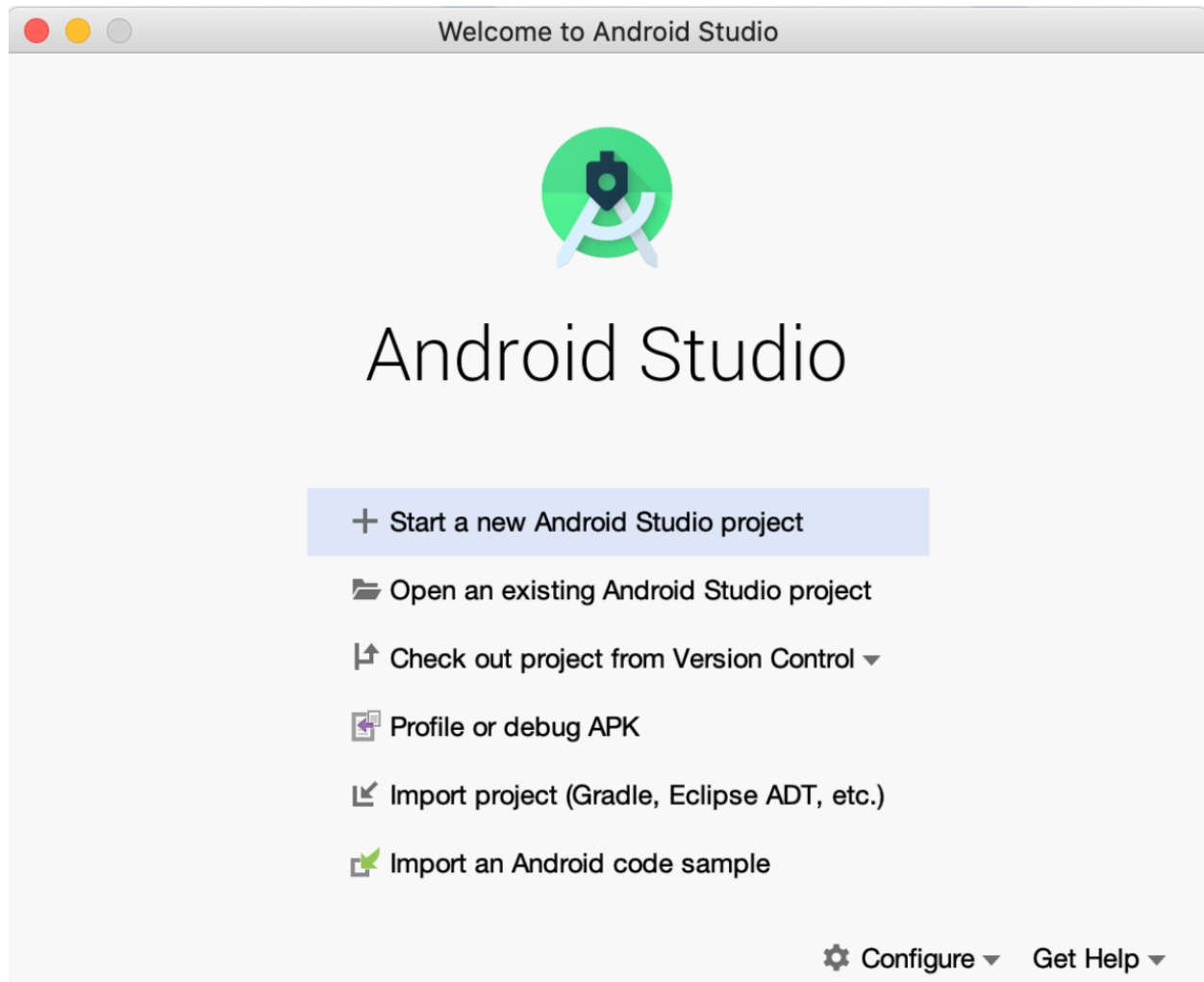
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



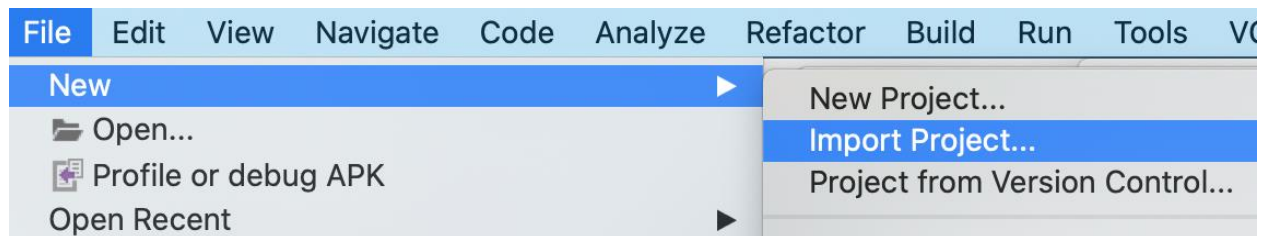
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it works as expected.

7. Browse the project files in the **Project** tool window to see how the app was implemented.

3. Add a RecyclerView

In this task, you will add a `RecyclerView` to the app to display the data stored in the database.

Add helper function to format price

Below is a screenshot of the final app.

Inventory

ITEM	PRICE	QUANTITY IN STOCK
Apples	\$43.00	54
Bananas	\$543.00	23
Blueberry	\$43.00	0
Honey	\$4.00	23
Oranges	\$45.00	123
Raspberry	\$6.00	94
Strawberry	\$5.00	5
Test	\$54.00	34
Tomatoes	\$5.00	32



Notice that the price is displayed in the currency format. To convert a double value to the desired currency format, you will add an extension function to the `Item` class.

Extension Functions

Kotlin provides an ability to extend a class with new functionality without having to inherit from the class or modify the existing definition of the class. That means you can add functions to an existing class without having to access its source code. This is done via special declarations called [extensions](#).

For example, you can write new functions for a class from a third-party library that you can't modify. Such functions are available for calling in the usual way, as if they were methods of the original class. These functions are called *extension functions*. (There are also *extension properties* that let you define new properties for existing classes, but these are outside the scope of this codelab.)

Extension functions don't actually modify the class, but allow you to use the dot-notation when calling the function on objects of that class.

For example, in the following code snippet you have a class called `Square`. This class has a property for the side and a function to calculate the area of the square. Notice the `Square.perimeter()` extension function, the function name is prefixed with the class it operates on. Inside the function, you can reference the public properties of the `Square` class.

Observe the extension function usage in the `main()` function. The created extension function, `perimeter()`, is called as a regular function inside that `Square` class.

Example:

```
class Square(val side: Double){
    fun area(): Double{
        return side * side;
    }
}

// Extension function to calculate the perimeter of the square
fun Square.perimeter(): Double{
    return 4 * side;
}

// Usage
fun main(args: Array<String>){
    val square = Square(5.5);
    val perimeterValue = square.perimeter()
    println("Perimeter: $perimeterValue")
    val areaValue = square.area()
    println("Area: $areaValue")
}
```

In this step, you will format the item price to a currency format string. In general, you don't want to change an entity class that represents data just to format the data (see [single responsibility principle](#)), so instead you'll add an extension function.

1. In `Item.kt`, below the class definition, add an extension function called `Item.getFormattedPrice()` that takes no parameters and returns a `String`. Notice the class name and the dot-notation in the function name.

```
fun Item.getFormattedPrice(): String =  
    NumberFormat.getCurrencyInstance().format(itemPrice)
```

Import `java.text.NumberFormat`, when prompted by Android Studio.

Add ListAdapter

In this step, you'll add a list adapter to the `RecyclerView`. Since you're familiar with implementing the adapter from previous codelabs, the instructions are summarized below. The completed `ItemListAdapter` file is at the end of this step for your convenience, and to help increase your understanding of the Room concepts in the codelab.

1. In the `com.example.inventory` package, add a Kotlin class named `ItemListAdapter`. Pass in a function called `onItemClicked()` as a constructor parameter that takes in an `Item` object as parameter.
2. Change the `ItemListAdapter` class signature to extend `ListAdapter`. Pass in the `Item` and `ItemListAdapter.ItemViewHolder` as parameters.
3. Add the constructor parameter `DiffCallback`; the `ListAdapter` will use this to figure out what changed in the list.
4. Override the required methods `onCreateViewHolder()` and `onBindViewHolder()`.
5. The `onCreateViewHolder()` method returns a new `ViewHolder` when `RecyclerView` needs one.
6. Inside the `onCreateViewHolder()` method, create a new `View`, inflate it from the `item_list_item.xml` layout file using the auto generated binding class, `ItemListItemBinding`.
7. Implement the `onBindViewHolder()` method. Get the current item using the method `getItem()`, passing the position.
8. Set the click listener on the `itemView`, call the function `onItemClicked()` inside the listener.
9. Define the `ItemViewHolder` class, extend it from `RecyclerView.ViewHolder`. Override the `bind()` function, pass in the `Item` object.
10. Define a companion object. Inside the companion object, define a `val` of the type `DiffUtil.ItemCallback<Item>()` called `DiffCallback`. Override the required methods `areItemsTheSame()` and `areContentsTheSame()`, and define them.

The finished class should look like the following:

```

package com.example.inventory

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.example.inventory.data.Item
import com.example.inventory.data.getFormattedPrice
import com.example.inventory.databinding.ItemListItemBinding

/**
 * [ListAdapter] implementation for the recyclerview.
 */

class ItemListAdapter(private val onItemClick: (Item) -> Unit) :
    ListAdapter<Item, ItemListAdapter.ItemViewHolder>(DiffCallback) {

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        ItemViewHolder {
        return ItemViewHolder(
            ItemListItemBinding.inflate(
                LayoutInflater.from(
                    parent.context
                )
            )
        )
    }

    override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {
        val current = getItem(position)
        holder.itemView.setOnClickListener {
            onItemClick(current)
        }
        holder.bind(current)
    }

    class ItemViewHolder(private var binding: ItemListItemBinding) :
        RecyclerView.ViewHolder(binding.root) {

        fun bind(item: Item) {

        }

    }

    companion object {
        private val DiffCallback = object : DiffUtil.ItemCallback<Item>() {
            override fun areItemsTheSame(oldItem: Item, newItem: Item):
                Boolean {
                return oldItem === newItem
            }

            override fun areContentsTheSame(oldItem: Item, newItem: Item):
                Boolean {
                return oldItem.itemName == newItem.itemName
            }
        }
    }
}

```

```
}  
}
```

Observe the inventory list screen from the finished app (the solution app from the end of this codelab). Notice that every list element displays the name of the inventory item, the price in currency format, and the current stock in hand. In the previous steps you used the `item_list_item.xml` layout file with three TextViews to create rows. In the next step, you will bind the entity details to these TextViews.

1:55



Inventory

ITEM	PRICE	QUANTITY IN STOCK
Apples	\$43.00	54
Bananas	\$1.00	23
Blueberry	\$43.00	0
Honey	\$4.00	23
Oranges	\$45.00	123
Raspberry	\$6.00	94
Strawberry	\$5.00	5
Tomatoes	\$5.00	32



11. In `ItemListAdapter.kt`, implement the `bind()` function in `ItemViewHolder` class. Bind the `itemName` `TextView` to `item.itemName`. Get the price in currency format using the `getFormattedPrice()` extension function, and bind it to the `itemPrice` `TextView`. Convert the `quantityInStock` value to `String`, and bind it to the `itemQuantity` `TextView`. The completed method should look like this:

```
fun bind(item: Item) {
    binding.apply {
        itemName.text = item.itemName
        itemPrice.text = item.getFormattedPrice()
        itemQuantity.text = item.quantityInStock.toString()
    }
}
```

When prompted by Android Studio, import
`com.example.inventory.data.getFormattedPrice`.

Use ListAdapter

In this task, you will update the `InventoryViewModel` and the `ItemListFragment` to display the item details on the screen using the list adapter you created in the previous step.

1. At the beginning of the class `InventoryViewModel`, create a `val` named `allItems` of the type `LiveData<List<Item>>` for the items from the database. Don't worry about the error, you will fix it soon.

```
val allItems: LiveData<List<Item>>
```

Import `androidx.lifecycle.LiveData` when prompted by the Android Studio.

2. Call `getItemDao()` on `itemDao` and assign it to `allItems`. The `getItemDao()` function returns a `Flow`. To consume the data as a `LiveData` value, use the [`asLiveData\(\)`](#) function. The finished definition should look like this:

```
val allItems: LiveData<List<Item>> = itemDao.getItemDao().asLiveData()
```

Import `androidx.lifecycle.asLiveData`, when prompted by the Android Studio.

3. In `ItemListFragment`, at the beginning of the class, declare a private immutable property called `viewModel` of the type `InventoryViewModel`. Use `by delegate` to hand off the property initialization to the `activityViewModels` class. Pass in the `InventoryViewModelFactory` constructor.

```
private val viewModel: InventoryViewModel by activityViewModels {
    InventoryViewModelFactory(
        (activity?.application as InventoryApplication).database.itemDao()
    )
}
```

Import `androidx.fragment.app.activityViewModels` when requested by the Android Studio.

4. Still within the `ItemListFragment`, scroll to function `onViewCreated()`. Below the call to `super.onViewCreated()`, declare a `val` named `adapter`. Initialize the new `adapter` property using the default constructor, `ItemListAdapter{}` passing in nothing.
5. Bind the newly created `adapter` to the `recyclerView` as follows:

```
val adapter = ItemListAdapter {  
}  
binding.recyclerView.adapter = adapter
```

6. Still inside `onViewCreated()`, after setting the `adapter`. Attach an observer on the `allItems` to listen for the data changes.
7. Inside the observer, call `submitList()` on the `adapter` and pass in the new list. This will update the `RecyclerView` with the new items on the list.

```
viewModel.allItems.observe(this.viewLifecycleOwner) { items ->  
    items.let {  
        adapter.submitList(it)  
    }  
}
```

8. Verify that the completed `onViewCreated()` method looks like the below. Run the app. Notice that the inventory list is displayed, if you items saved in your app database. Add some inventory items to the app database if the list is empty.

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
    super.onViewCreated(view, savedInstanceState)  
  
    val adapter = ItemListAdapter {  
    }  
    binding.recyclerView.adapter = adapter  
    viewModel.allItems.observe(this.viewLifecycleOwner) { items ->  
        items.let {  
            adapter.submitList(it)  
        }  
    }  
    binding.recyclerView.layoutManager = LinearLayoutManager(this.context)  
    binding.floatingActionButton.setOnClickListener {  
        val action =  
        ItemListFragmentDirections.actionItemListFragmentToAddItemFragment(  
            getString(R.string.add_fragment_title)  
        )  
        this.findNavController().navigate(action)  
    }  
}
```

1:55



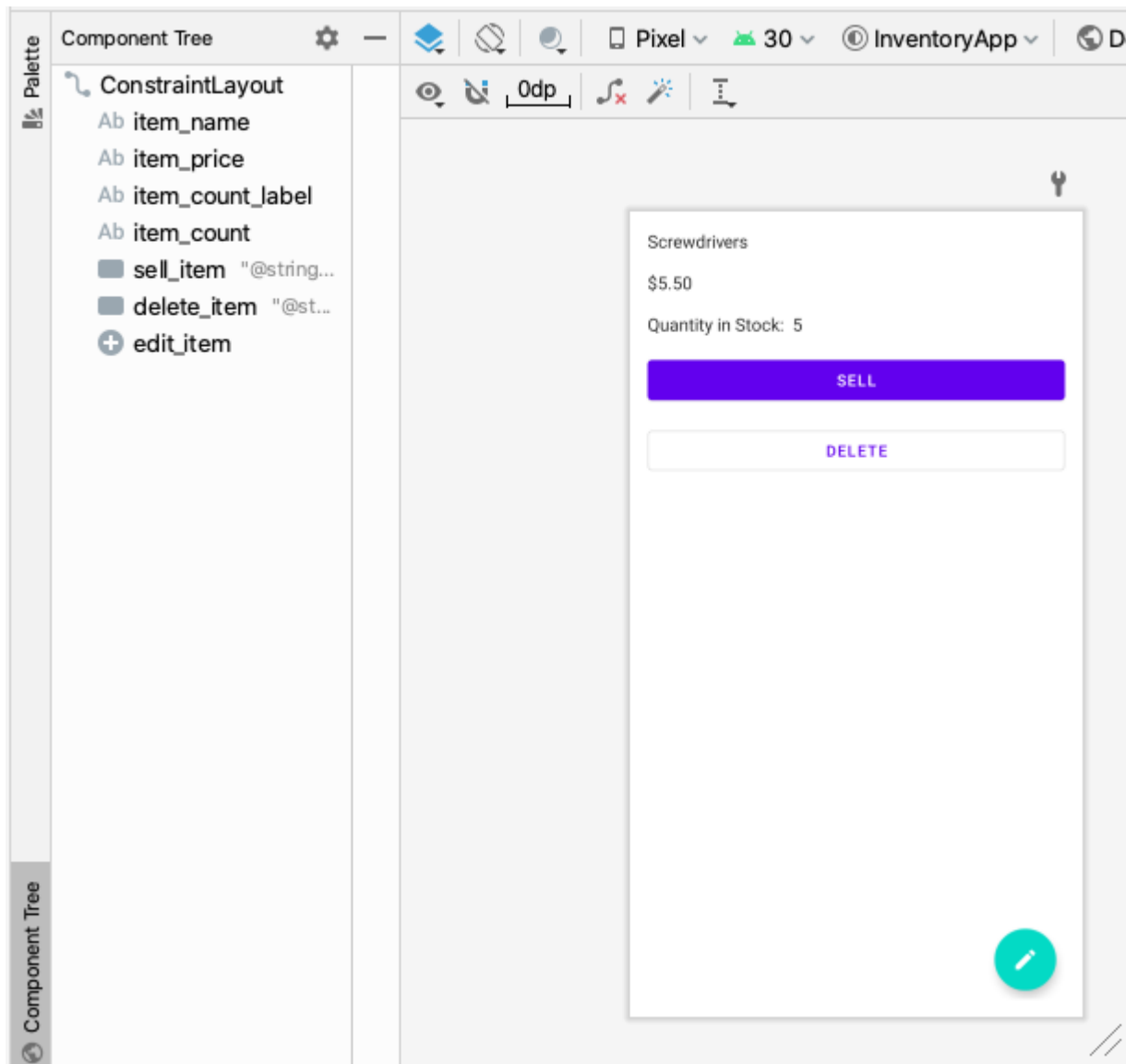
Inventory

ITEM	PRICE	QUANTITY IN STOCK
Apples	\$43.00	54
Bananas	\$1.00	23
Blueberry	\$43.00	0
Honey	\$4.00	23
Oranges	\$45.00	123
Raspberry	\$6.00	94
Strawberry	\$5.00	5
Tomatoes	\$5.00	32



4. Display item details

In this task, you will read and display the entity details on the **Item Details** screen. You will use the primary key (the item id) to read the details, such as name, price and quantity from the inventory app database and display them on the **Item Details** screen using the `fragment_item_detail.xml` layout file. The layout file `fragment_item_detail.xml` is predesigned for you and contains three TextViews that display the item details.



You will implement the following steps in this task:

- Add a click handler to the RecyclerView to navigate the app to the **Item Details** screen.
- In the `ItemListFragment` fragment, retrieve the data from the database and display.

- Bind the TextViews to the ViewModel data.

Add a click handler

1. In `ItemListFragment`, scroll to the `onViewCreated()` function to update the adapter definition.
2. Add a lambda as a constructor parameter to the `ItemListAdapter{}`.

```
val adapter = ItemListAdapter {  
}
```

3. Inside the lambda, create a `val` called `action`. You will fix the initialization error soon.

```
val adapter = ItemListAdapter {  
    val action  
}
```

4. Call `actionItemListFragmentToItemDetailFragment()` method on the `ItemListFragmentDirections` passing in the `item id`. Assign the returned `NavDirections` object to `action`.

```
val adapter = ItemListAdapter {  
    val action  
    = ItemListFragmentDirections.actionItemListFragmentToItemDetailFragment(it  
    .id)  
}
```

5. Below the `action` definition, retrieve a `NavController` instance using `this.findNavController()` and call `navigate()` on it passing in the `action`. The adapter definition should look like this:

```
val adapter = ItemListAdapter {  
    val action  
    = ItemListFragmentDirections.actionItemListFragmentToItemDetailFragment(it.  
    id)  
    this.findNavController().navigate(action)  
}
```

6. Run the app. Click on an item in the `RecyclerView`. The app navigates to the **Item Details** screen. Notice that the details are blank. Tap on the buttons, nothing happens.



Item Details

Quantity in Stock:

SELL

DELETE



In later steps you will display the entity details on the **Item Details** screen and add functionality to sell and delete buttons.

Retrieve item details

In this step, you will add a new function to the `InventoryViewModel`, to retrieve the item details from the database based on the item `id`. In the next step, you will use this function to display the entity details on the **Item Details** screen.

1. In `InventoryViewModel`, add a function named `retrieveItem()` that takes an `Int` for the item `id` and returns a `LiveData<Item>`. You will fix the return expression error soon.

```
fun retrieveItem(id: Int): LiveData<Item> {  
}
```

2. Inside the new function, call `getItem()` on the `itemDao`, passing in the parameter `id`. The `getItem()` function returns a `Flow`. To consume the `Flow` value as `LiveData` call `asLiveData()` function and use this as the return of `retrieveItem()` function. The completed function should look like the following:

```
fun retrieveItem(id: Int): LiveData<Item> {  
    return itemDao.getItem(id).asLiveData()  
}
```

Bind data to the TextViews

In this step, you will create a `ViewModel` instance in the `ItemDetailFragment` and bind the `ViewModel` data to the `TextViews` in the **Item Details** screen. You will also attach an observer to the data in the `ViewModel` to keep your inventory list updated on the screen, if underlying data in the database changes.

1. In `ItemDetailFragment`, add a mutable property called `item` of the type `Item` entity. You will use this property to store information about a single entity. This property will be initialized later, so prefix it with `lateinit`.

```
lateinit var item: Item
```

Import `com.example.inventory.data.Item`, when prompted by the Android Studio.

2. At the beginning of the class `ItemDetailFragment`, declare a private immutable property called `viewModel` of the type `InventoryViewModel`. Use `by delegate` to hand off the property initialization to the `activityViewModels` class. Pass in the `InventoryViewModelFactory` constructor.

```
private val viewModel: InventoryViewModel by activityViewModels {  
    InventoryViewModelFactory(  

```

```

        (activity?.application as InventoryApplication).database.itemDao()
    )
}

```

Import `androidx.fragment.app.activityViewModels`, if prompted by Android Studio.

3. Still in `ItemDetailFragment`, create a private function called `bind()` that takes an instance of the `Item` entity as the parameter and returns nothing.

```

private fun bind(item: Item) {
}

```

4. Implement the `bind()` function, this is similar to what you have done in the `ItemListAdapter`. Set the `text` property of `itemName` `TextView` to `item.itemName`. Call `getFormattedPrice()` on the `item` property to format the price value, and set it to the `text` property of `itemPrice` `TextView`. Convert the `quantityInStock` to `String` and set it to the `text` property of `itemQuantity` `TextView`.

```

private fun bind(item: Item) {
    binding.itemName.text = item.itemName
    binding.itemPrice.text = item.getFormattedPrice()
    binding.itemCount.text = item.quantityInStock.toString()
}

```

5. Update the `bind()` function to use the `apply{} scope` function to the code block as shown below.

```

private fun bind(item: Item) {
    binding.apply {
        itemName.text = item.itemName
        itemPrice.text = item.getFormattedPrice()
        itemCount.text = item.quantityInStock.toString()
    }
}

```

6. Still in `ItemDetailFragment`, override `onViewCreated()`.

```

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
}

```

7. In one of the previous steps, you passed `item id` as a navigation argument to `ItemDetailFragment` from the `ItemListFragment`. Inside `onViewCreated()`, below the call to the super class function, create an immutable variable called `id`. Retrieve and assign the navigation argument to this new variable.

```

val id = navigationArgs.itemId

```

8. Now you'll use this `id` variable to retrieve the item details. Still inside `onViewCreated()`, call the `retrieveItem()` function on the `viewModel` passing in the `id`. Attach an observer to the returned value passing in the `viewLifecycleOwner` and a lambda.

```
viewModel.retrieveItem(id).observe(this.viewLifecycleOwner) {  
    }
```

9. Inside the lambda, pass in `selectedItem` as the parameter which contains the `Item` entity retrieved from the database. In the lambda function body, assign `selectedItem` value to `item`. Call `bind()` function passing in the `item`. The completed function should look like the following.

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
    super.onViewCreated(view, savedInstanceState)  
    val id = navigationArgs.itemId  
    viewModel.retrieveItem(id).observe(this.viewLifecycleOwner) { selectedItem  
->  
        item = selectedItem  
        bind(item)  
    }  
}
```

10. Run the app. Click on any list element on the **Inventory** screen, **Item Details** screen is displayed. Notice that now the screen is not blank any more, it displays the entity details retrieved from the inventory database.

1:55 ⚙

Inventory	
ITEM	PRICE
Apples	\$43
Bananas	\$1
Bluberry	\$43
Honey	\$4
Oranges	\$45
Rasberry	\$6
Strawberry	\$5
Tomatoes	\$5

◀ ●

1:55 ⚙

← Item Details

Bananas

\$1.00

Quantity in Stock: 23

SELL

DELETE

◀ ●

11. Tap on the **Sell**, **Delete**, and FAB buttons. Nothing happens! In next tasks, you'll implement the functionality of these buttons.

5. Implement sell item

In this task, you will extend the features of the app, implement sell functionality. Here is a high level gist of the instructions for this step.

- Add a function in the ViewModel to update an entity
- Create a new method to reduce the quantity and update the entity in the app database.
- Attach a click listener to the **Sell** button
- Disable the **Sell** button if the quantity is zero.

Let's code:

1. In `InventoryViewModel`, add a private function called `updateItem()` that takes an instance of the entity class, `Item` and returns nothing.

```
private fun updateItem(item: Item) {  
}
```

2. Implement the new method, `updateItem()`. To call `update()` suspend method from the `ItemDao` class, launch a coroutine using the `viewModelScope`. Inside the launch block, make a call to the `update()` function on `itemDao` passing in the `item`. Your completed method should look like the following.

```
private fun updateItem(item: Item) {  
    viewModelScope.launch {  
        itemDao.update(item)  
    }  
}
```

3. Still inside the `InventoryViewModel`, add another method called `sellItem()` that takes an instance of the `Item` entity class and returns nothing.

```
fun sellItem(item: Item) {  
}
```

4. Inside the `sellItem()` function, add an `if` condition to check whether the `item.quantityInStock` is greater than 0.

```
fun sellItem(item: Item) {  
    if (item.quantityInStock > 0) {  
    }  
}
```

Inside the `if` block you will use `copy()` function for Data class to update the entity.

Data class: copy()

The `copy()` function is provided by default to all the instances of data classes. This function is used to copy an object for changing some of its properties, but keeping the rest of the properties unchanged.

For example, consider the `User` class and its instance `jack` as shown below. If you want to create a new instance with only updating the `age` property, its implementation would be as follows:

Example

```
// Data class
data class User(val name: String = "", val age: Int = 0)

// Data class instance
val jack = User(name = "Jack", age = 1)

// A new instance is created with its age property changed, rest of the
properties unchanged.
val olderJack = jack.copy(age = 2)
```

5. Back to the `sellItem()` function in the `InventoryViewModel`. Inside the `if` block, create a new immutable property called `newItem`. Call `copy()` function on the `item` instance passing in the updated `quantityInStock`, that is decreasing the stock by 1.

```
val newItem = item.copy(quantityInStock = item.quantityInStock - 1)
```

6. Below the definition of the `newItem`, make a call to the `updateItem()` function passing in the new updated entity, that is `newItem`. The completed method should look like the following.

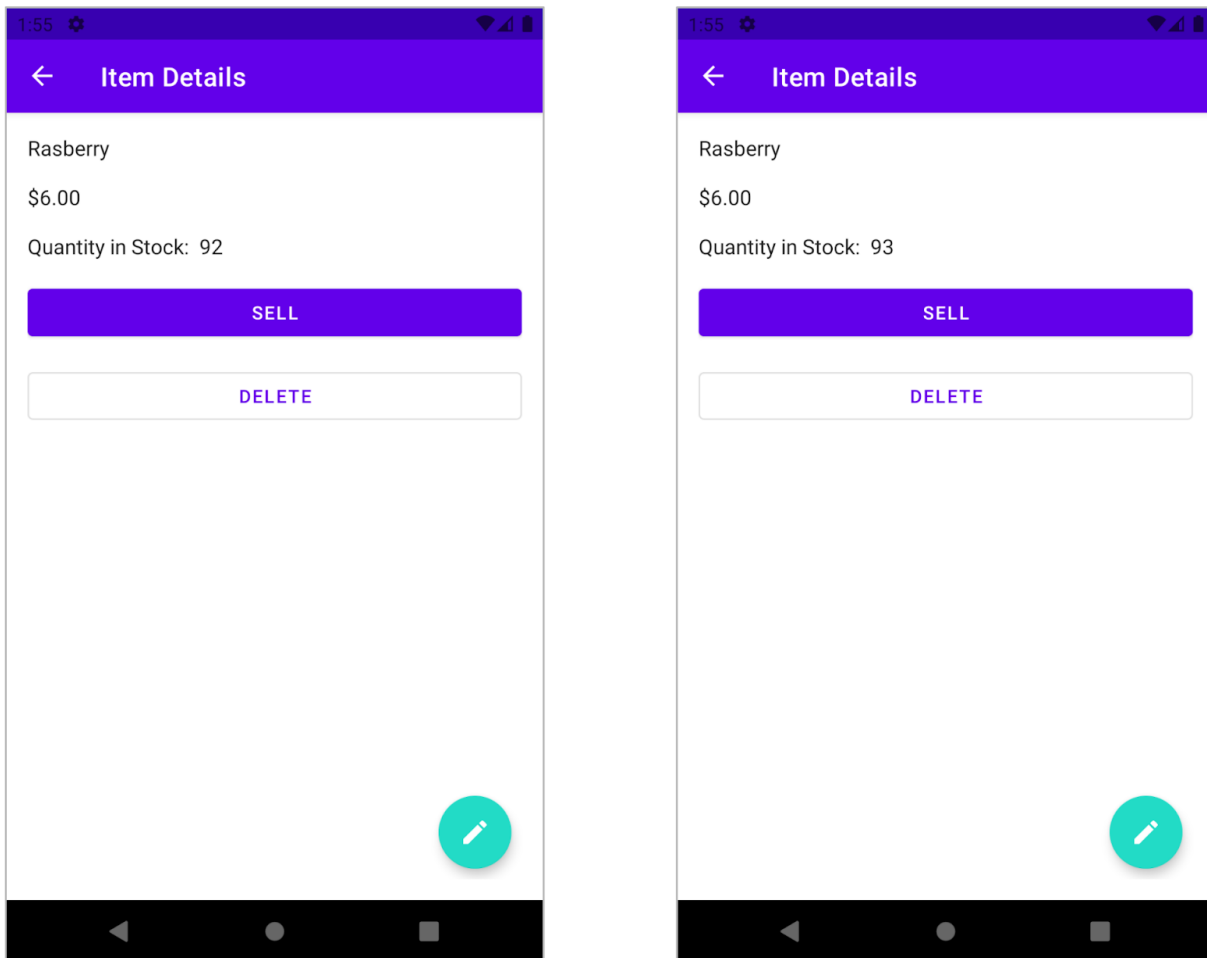
```
fun sellItem(item: Item) {
    if (item.quantityInStock > 0) {
        // Decrease the quantity by 1
        val newItem = item.copy(quantityInStock = item.quantityInStock - 1)
        updateItem(newItem)
    }
}
```

7. To add the selling stock feature, go to `ItemDetailFragment`. Scroll to the end of the `bind()` function. Inside the `apply` block, set a click listener to the **Sell** button and call the `sellItem()` function on `viewModel`.

```
private fun bind(item: Item) {
    binding.apply {
        ...
        sellItem.setOnClickListener { viewModel.sellItem(item) }
    }
}
```

```
}  
}
```

8. Run the app. On the **Inventory** screen click on a list element with quantity greater than zero. The **Item Details** screen will be displayed. Tap **Sell** button, notice the quantity value is decreased by one.



9. In the **Item Details** screen make the quantity 0 by continuously tapping the **Sell** button. (Tip: Select an entity with less stock or create a new one with less quantity). Once the quantity is zero, tap the **Sell** button. There will be no visual change. This is because your function `sellItem()` checks if the quantity is greater than zero, before updating the quantity.



Item Details

Blueberry

\$43.00

Quantity in Stock: 0

SELL

DELETE



10. To give users better feedback, you might want to disable the **Sell** button when there is no item to sell. In `InventoryViewModel`, add a function to check if the quantity is greater than 0. Name the function `isStockAvailable()`, that takes an `Item` instance and returns a `Boolean`.

```
fun isStockAvailable(item: Item): Boolean {  
    return (item.quantityInStock > 0)  
}
```

11. Go to `ItemDetailFragment`, scroll to the `bind()` function. Inside the `apply` block, call the `isStockAvailable()` function on `viewModel` passing in the `item`. Set the return value to `isEnabled` property of the **Sell** button. Your code should look something like this.

```
private fun bind(item: Item) {  
    binding.apply {  
        ...  
        sellItem.isEnabled = viewModel.isStockAvailable(item)  
        sellItem.setOnClickListener { viewModel.sellItem(item) }  
    }  
}
```

12. Run your app, notice that the **Sell** button is disabled when the quantity in stock is zero. Congratulations on implementing the sell item feature to your app.



Item Details

Blueberry

\$43.00

Quantity in Stock: 0

SELL

DELETE



Delete item entity

Similar to the previous task, you will extend the features of your app further by implementing delete functionality. Here are the high-level instructions for this step, it's much easier than implementing the sell feature.

- Add a function in the ViewModel to delete an entity from the database
- Add a new method in the `ItemDetailFragment` to call the new delete function and handle navigation.
- Attach a click listener to the **Delete** button.

Let's continue to code:

1. In `InventoryViewModel`, add a new function called `deleteItem()`, which takes an instance of the `Item` entity class called `item` and returns nothing. Inside the `deleteItem()` function, launch a coroutine with `viewModelScope`. Inside the launch block call the `delete()` method on `itemDao` passing in the `item`.

```
fun deleteItem(item: Item) {  
    viewModelScope.launch {  
        itemDao.delete(item)  
    }  
}
```

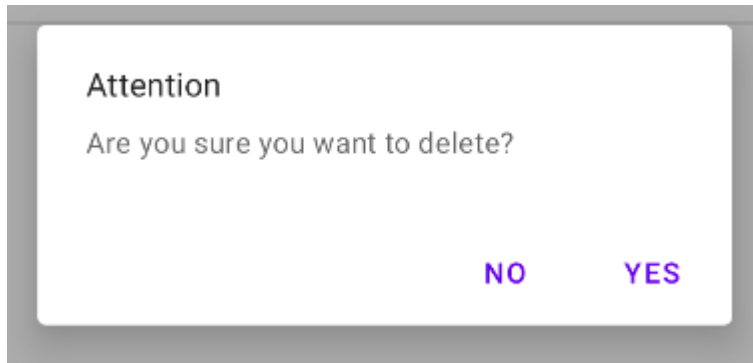
2. In `ItemDetailFragment`, scroll to the beginning of the `deleteItem()` function. Call `deleteItem()` on the `viewModel`, pass in the `item`. The `item` instance contains the entity currently displayed on the **Item Details** screen. Your completed method should look like this.

```
private fun deleteItem() {  
    viewModel.deleteItem(item)  
    findNavController().navigateUp()  
}
```

3. Still within `ItemDetailFragment`, scroll to the `showConfirmationDialog()` function. This function is given for you as part of the starter code. This method displays an alert dialog to get the user's confirmation before deleting the item and calls `deleteItem()` function when the positive button is tapped.

```
private fun showConfirmationDialog() {  
    MaterialAlertDialogBuilder(requireContext())  
        ...  
        .setPositiveButton(getString(R.string.yes)) { _, _ ->  
            deleteItem()  
        }  
        .show()  
}
```

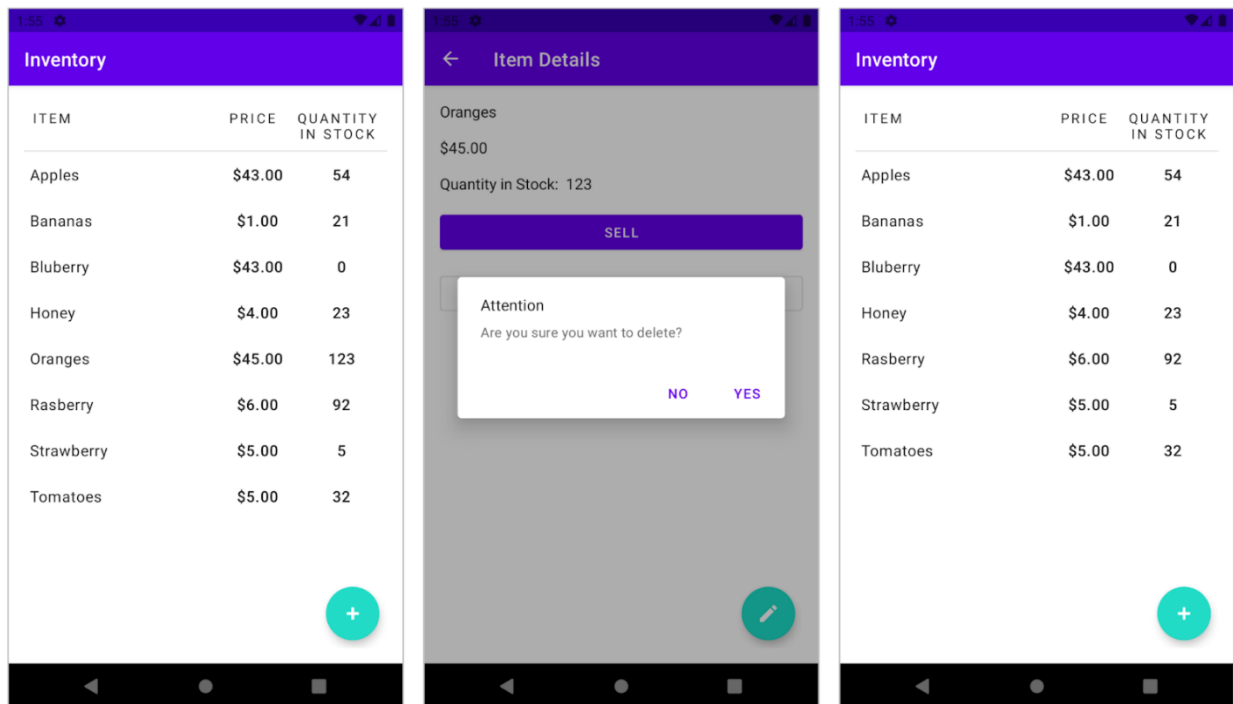
The `showConfirmationDialog()` function displays a alert dialog which looks like the following:



4. In `ItemDetailFragment`, at the end of `bind()` function, inside the `apply` block, set the click listener to the delete button. Call `showConfirmationDialog()` inside the click listener lambda.

```
private fun bind(item: Item) {
    binding.apply {
        ...
        deleteItem.setOnClickListener { showConfirmationDialog() }
    }
}
```

5. Run your app! Select a list element on the Inventory list screen, in the **Item Details** screen tap **Delete** button. Tap Yes, the app navigates back to the Inventory screen. Notice that the entity you deleted is no longer in the app database. Congratulations on implementing the delete feature.



Edit item entity

Similar to the previous tasks, in this task you will add another feature enhancement to the app. You will implement the edit item entity.

Here is a quick run through of the steps to edit an entity in the app database:

- Reuse the **Add Item** screen by updating the fragment title to Edit Item.
- Add click listener to the FAB, to navigate to the **Edit Item** screen.
- Populate the TextViews with the entity details.
- Update the entity in the database using Room.

Add click listener to the FAB

1. In `ItemDetailFragment`, add a new `private` function called `editItem()` that takes no parameters and returns nothing. In the next step, you will be reusing the `fragment_add_item.xml`, by updating the screen title to **Edit Item**. To achieve this you will send the fragment title string along with item id as part of the action.

```
private fun editItem() {  
}
```

After you update the fragment title the **Edit Item** screen should look like the following.



Edit Item

Item Name *

Item Price *

Quantity in Stock *

SAVE

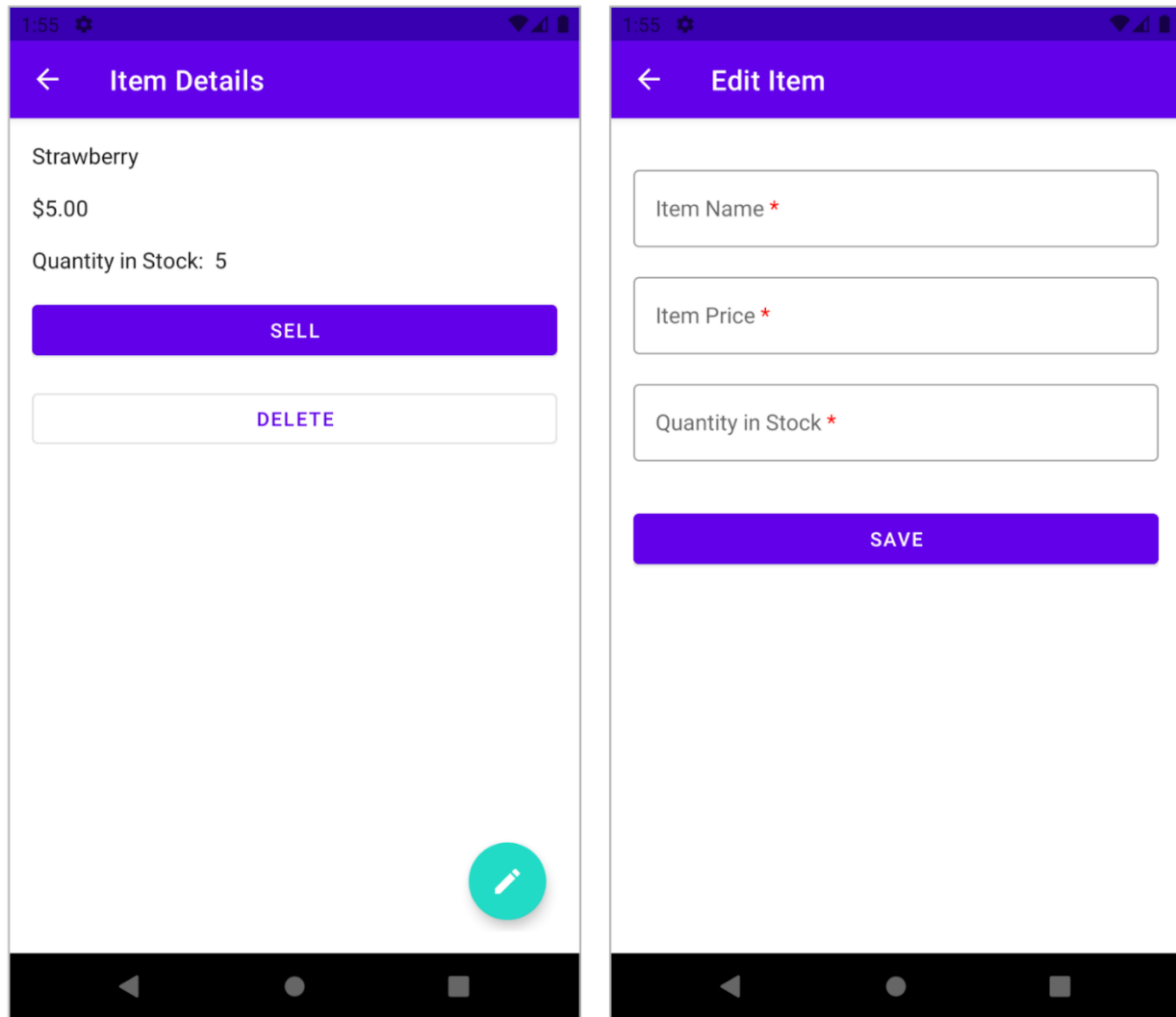
2. Inside `editItem()` function, create an immutable variable called `action`. Make a call to `actionItemDetailFragmentToAddItemFragment()` on `ItemDetailFragmentDirections` passing in title string, `edit_fragment_title` and the item id. Assign the returned value to `action`. Below the definition of `action`, call `this.findNavController().navigate()` passing in the `action` to navigate to the **Edit Item** screen.

```
private fun editItem() {  
    val action =  
        ItemDetailFragmentDirections.actionItemDetailFragmentToAddItemFragment(  
            getString(R.string.edit_fragment_title),  
            item.id  
        )  
    this.findNavController().navigate(action)  
}
```

3. Still within `ItemDetailFragment`, scroll to the `bind()` function. Inside the `apply` block, set the click listener to the FAB, call the `editItem()` function from the lambda to navigate to the **Edit Item** screen.

```
private fun bind(item: Item) {  
    binding.apply {  
        ...  
        editItem.setOnClickListener { editItem() }  
    }  
}
```

4. Run the app. Go to the **Item Details** screen. Click on FAB. Notice the title of the screen is updated to Edit Item, but all text fields are empty. In the next step, you'll fix this.



Populate TextViews

In this step, you will populate the text fields in the **Edit Item** screen with the entity details. Since we are using the `Add Item` screen you will add new functions to the Kotlin file,

`AddItemFragment.kt`.

1. In `AddItemFragment`, add a new `private` function to bind the text fields with entity details. Name the function `bind()` that takes in instance of the `Item` entity class and returns nothing.

```
private fun bind(item: Item) {  
}
```

2. Implementation of the `bind()` function is very similar to what you had done earlier in `ItemDetailFragment`. Inside the `bind()` function, round the price to two decimal places using the `format()` function and assign it to a `val` named `price`, as shown below.

```
val price = "%.2f".format(item.itemPrice)
```

3. Below the price definition, use the `apply` scope function on the binding property as shown below.

```
binding.apply {  
}
```

4. Inside the `apply` scope function code block, Set `item.itemName` to the text property of the `itemName`. Use `setText()` function and pass in `item.itemName` string and `TextView.BufferType.SPANNABLE` as `BufferType`.

```
binding.apply {  
    itemName.setText(item.itemName, TextView.BufferType.SPANNABLE)  
}
```

Import `android.widget.TextView`, if prompted by Android Studio.

5. Similar to the above step, set the text property of the price `EditText` as shown below. For setting text property of quantity `EditText` remember to convert the `item.quantityInStock` to `String`. Your completed function should look like this.

```
private fun bind(item: Item) {  
    val price = "%.2f".format(item.itemPrice)  
    binding.apply {  
        itemName.setText(item.itemName, TextView.BufferType.SPANNABLE)  
        itemPrice.setText(price, TextView.BufferType.SPANNABLE)  
        itemCount.setText(item.quantityInStock.toString(),  
TextView.BufferType.SPANNABLE)  
    }  
}
```

6. Still inside the `AddItemFragment`, scroll to the `onViewCreated()` function. After the call to the super class function. Create a `val` called `id` and retrieve `itemId` from the navigation arguments.

```
val id = navigationArgs.itemId
```

7. Add an `if-else` block with a condition to check whether `id` is greater than zero and move the **Save** button click listener into the `else` block. Inside the `if` block retrieve the entity using the `id` and add an observer on it. Inside the observer, update the `item` property and call `bind()` passing in the `item`. The complete function is provided for you to copy-paste. It is simple and easy to understand; you are left to decipher it on your own.

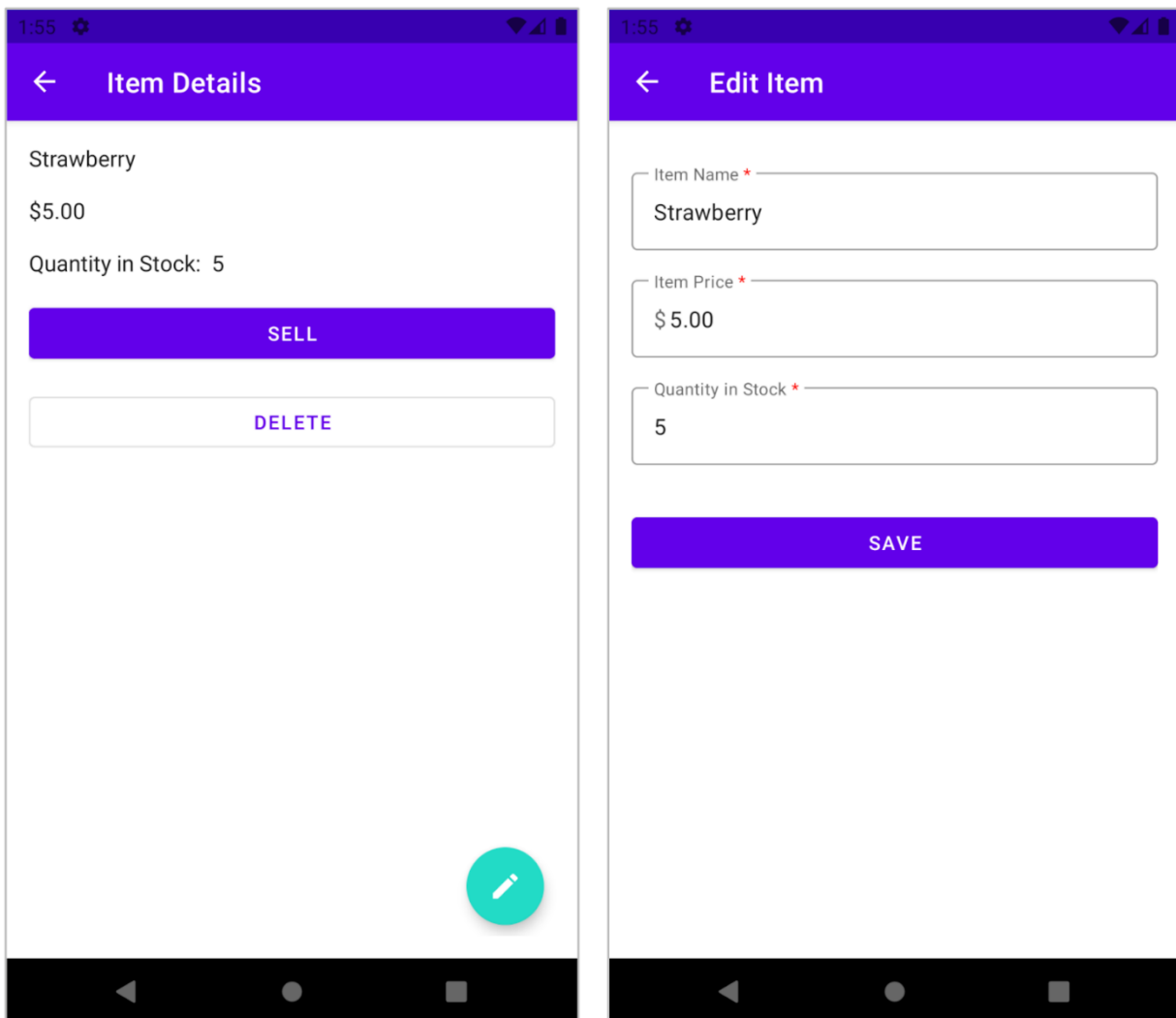
```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {  
    super.onViewCreated(view, savedInstanceState)  
    val id = navigationArgs.itemId  
    if (id > 0) {  
        viewModel.retrieveItem(id).observe(this.viewLifecycleOwner) {  
selectedItem ->  
}
```

```

        item = selectedItem
        bind(item)
    }
} else {
    binding.saveAction.setOnClickListener {
        addNewItem()
    }
}
}
}

```

- Run the app, Goto **Item Details**, tap + FAB. Notice the fields are filled with the item details. Edit the stock quantity or any other field and tap save button. Nothing happens! This is because you are not updating the entity in the app database. You will fix this soon.



Update the entity using Room

In this final task, add the final pieces of the code to implement the update functionality. You will define the necessary functions in the ViewModel and use them in the `AddItemFragment`.

It's coding time again!

1. In `InventoryViewModel`, add a private function called `getUpdatedItemEntry()` that takes in an `Int`, and three strings for the entity details named `itemName`, `itemPrice` and `itemCount`. Return an instance of the `Item` from the function. Code is given for your reference.

```
private fun getUpdatedItemEntry(  
    itemId: Int,  
    itemName: String,  
    itemPrice: String,  
    itemCount: String  
): Item {  
}
```

2. Inside the `getUpdatedItemEntry()` function create an `Item` instance using the function parameters, as shown below. Return the `Item` instance from the function.

```
private fun getUpdatedItemEntry(  
    itemId: Int,  
    itemName: String,  
    itemPrice: String,  
    itemCount: String  
): Item {  
    return Item(  
        id = itemId,  
        itemName = itemName,  
        itemPrice = itemPrice.toDouble(),  
        quantityInStock = itemCount.toInt()  
    )  
}
```

3. Still inside the `InventoryViewModel`, add another function named `updateItem()`. This function also takes an `Int` and three strings for the entity details and returns nothing. Use the variable names from the following code snippet.

```
fun updateItem(  
    itemId: Int,  
    itemName: String,  
    itemPrice: String,  
    itemCount: String  
) {  
}
```

4. Inside the `updateItem()` function make a call to the `getUpdatedItemEntry()` function passing in the entity information, which are passed in as function parameters, as shown below. Assign the returned value to an immutable variable called `updatedItem`.

```
val updatedItem = getUpdatedItemEntry(itemId, itemName, itemPrice, itemCount)
```

5. Just below the call to the `getUpdatedItemEntry()` function, make a call to the `updateItem()` function passing in the `updatedItem`. The completed function looks like this:

```
fun updateItem(  
    itemId: Int,  
    itemName: String,  
    itemPrice: String,  
    itemCount: String  
) {  
    val updatedItem = getUpdatedItemEntry(itemId, itemName, itemPrice,  
itemCount)  
    updateItem(updatedItem)  
}
```

6. Go back to `AddItemFragment`, add a private function called `updateItem()` with no parameters and return nothing. Inside the function add an `if` condition to validate the user input by calling the function `isEntryValid()`.

```
private fun updateItem() {  
    if (isEntryValid()) {  
    }  
}
```

7. Inside the `if` block, make a call to `viewModel.updateItem()` passing the entity details. Use the `itemId` from the navigation arguments, and the other entity details like name, price and quantity from the `EditTexts` as shown below.

```
viewModel.updateItem(  
    this.navigationArgs.itemId,  
    this.binding.itemName.text.toString(),  
    this.binding.itemPrice.text.toString(),  
    this.binding.itemCount.text.toString()  
)
```

8. Below the `updateItem()` function call, define an `val` called `action`. Call `actionAddItemFragmentToItemListFragment()` on `AddItemFragmentDirections` and assign the returned value to `action`. Navigate to `ItemListFragment`, call `findNavController().navigate()` passing in the `action`.

```
private fun updateItem() {  
    if (isEntryValid()) {  
        viewModel.updateItem(  
            this.navigationArgs.itemId,  
            this.binding.itemName.text.toString(),  
            this.binding.itemPrice.text.toString(),  
            this.binding.itemCount.text.toString()  
        )  
        val action =  
AddItemFragmentDirections.actionAddItemFragmentToItemListFragment()  
        findNavController().navigate(action)
```

```

    }
}

```

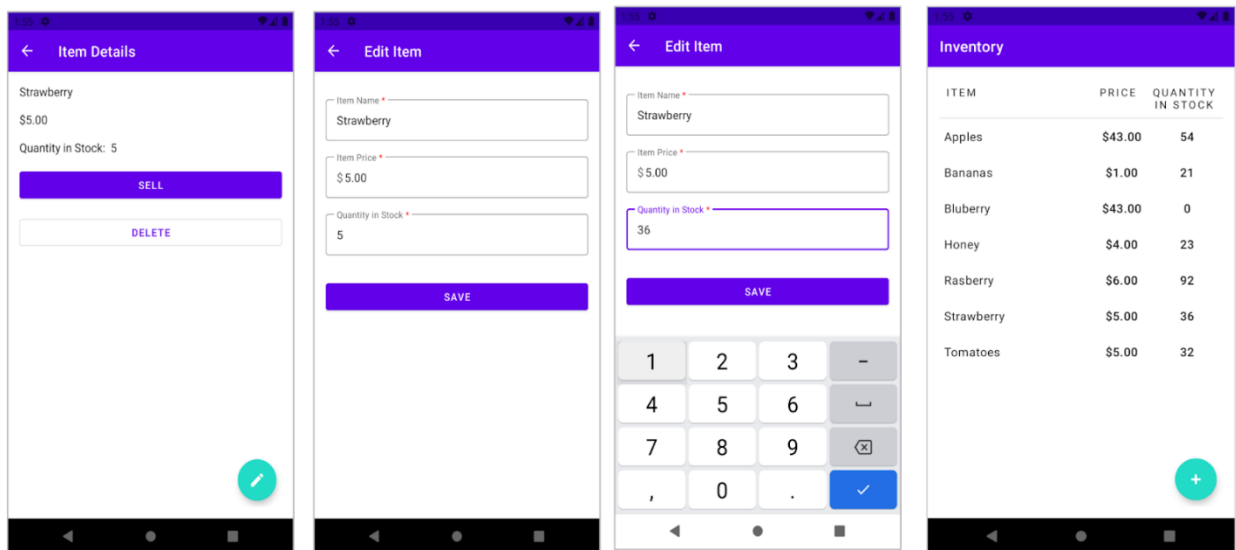
- Still within `AddItemFragment`, scroll to the `bind()` function. Inside the `binding.apply` scope function block set the click listener for the **Save** button. Make a call to the `updateItem()` function inside the lambda as shown below.

```

private fun bind(item: Item) {
    ...
    binding.apply {
        ...
        saveAction.setOnClickListener { updateItem() }
    }
}

```

- Run the app! Try editing inventory items; you should be able to edit any item in the Inventory app database.



Congratulations on creating your first app to use Room for managing the app database!

6. Solution code

The solution code for this codelab is in the GitHub repo and branch shown below.

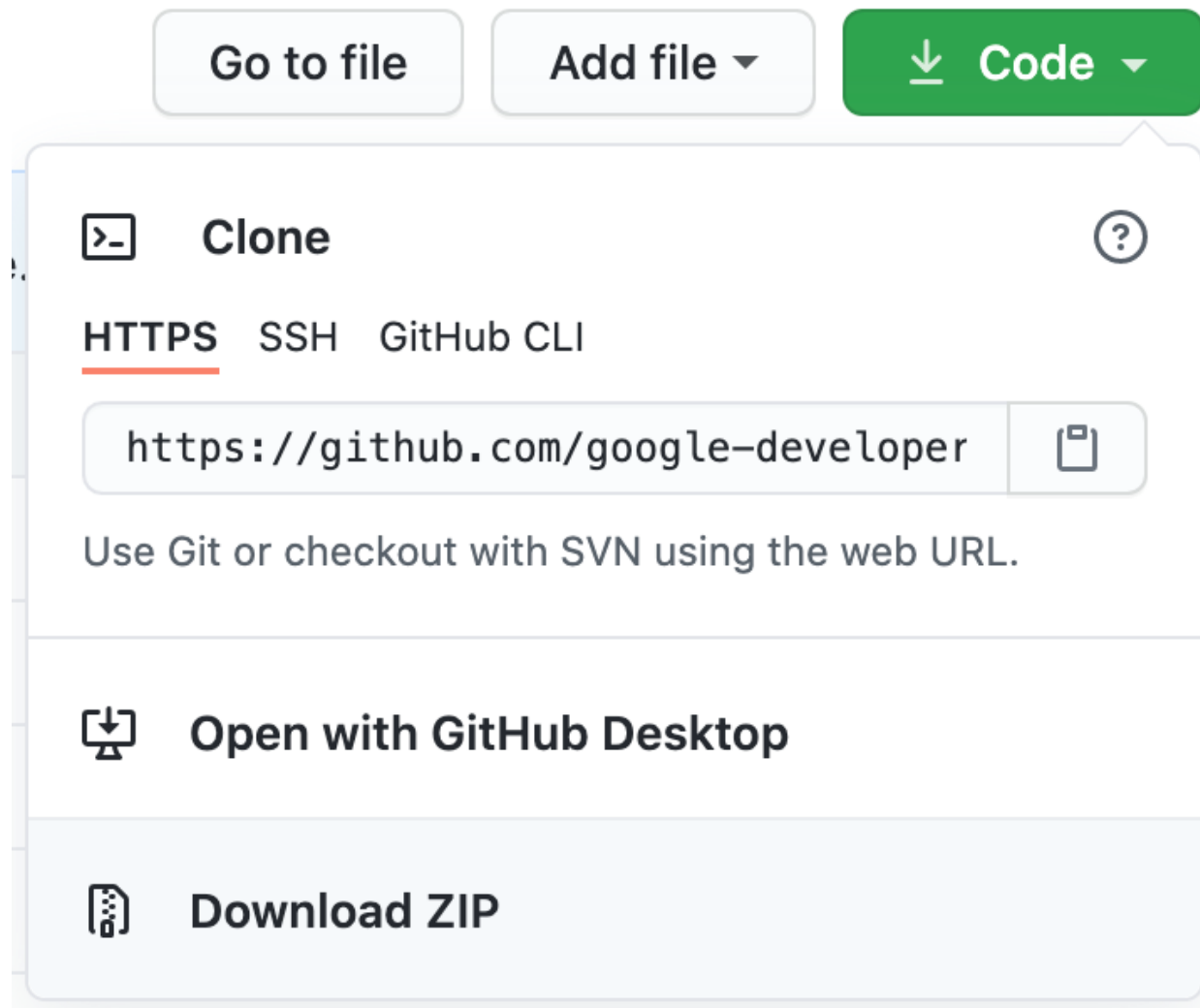
Solution Code URL: <https://github.com/google-developer-training/android-basics-kotlin-inventory-app>

Branch name: `main`

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

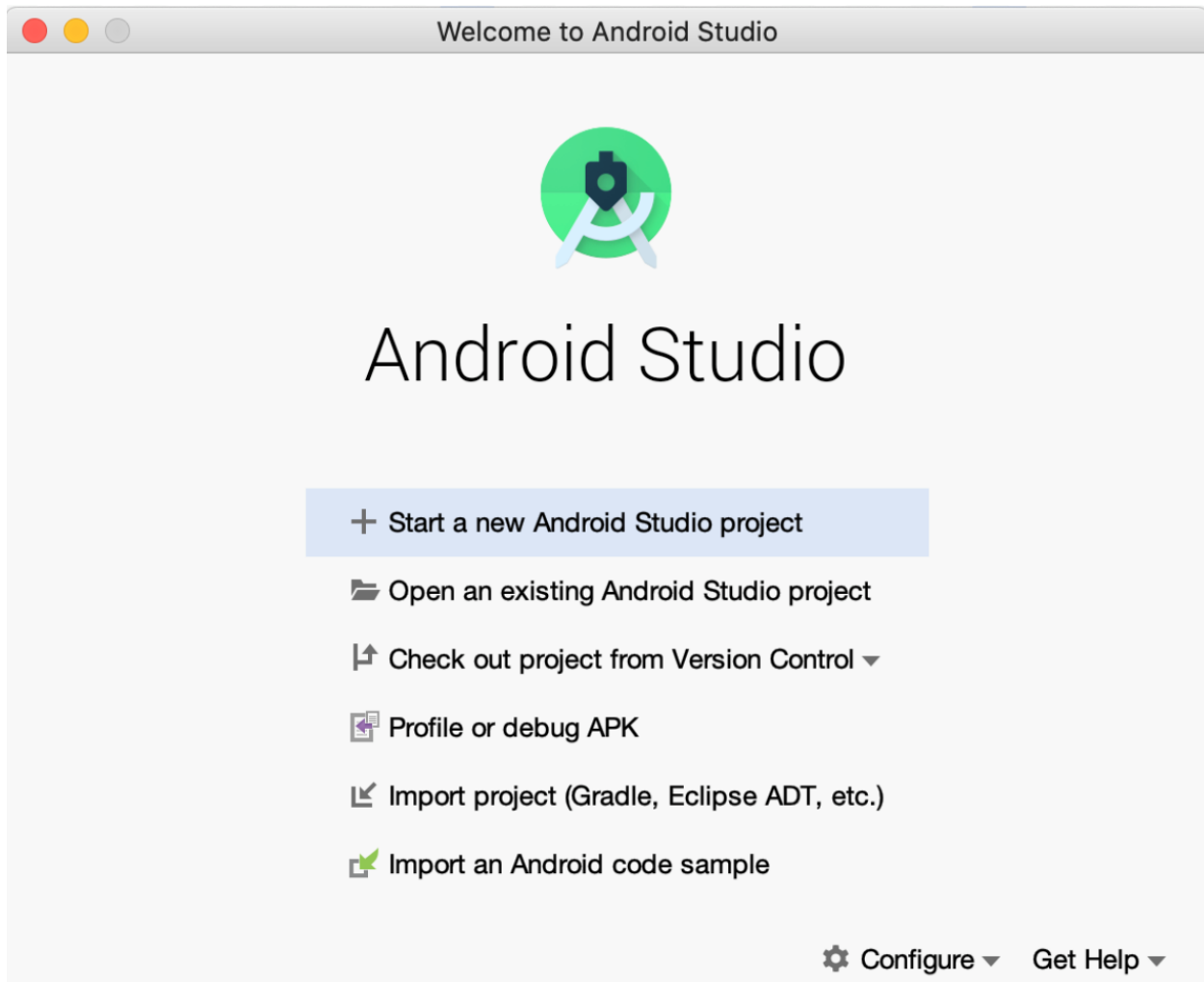
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



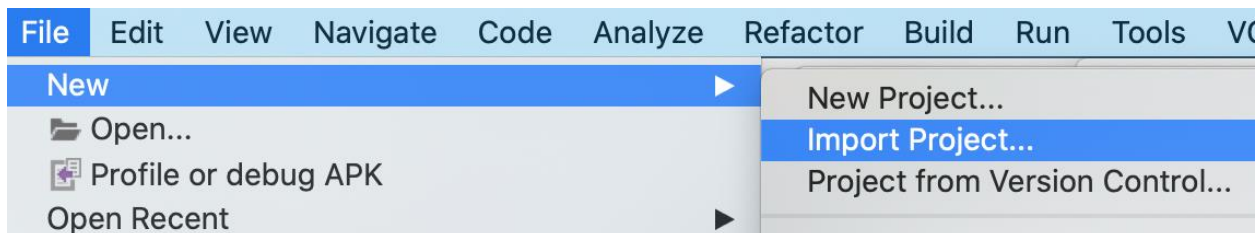
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.

5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it works as expected.
7. Browse the project files in the **Project** tool window to see how the app was implemented.

7. Summary

- Kotlin provides an ability to extend a class with new functionality without having to inherit from the class or modify the existing definition of the class. This is done via special declarations called [*extensions*](#).
- To consume the `Flow` data as a `LiveData` value, use the `asLiveData()` function.
- The `copy()` function is provided by default to all the instances of data classes. It lets you copy an object and change some of its properties, while keeping the rest of its properties unchanged.

8. Learn more

Android Developer Documentation

- [Pass data between destinations](#)
- [Android String](#)
- [Android Formatter](#)
- [Debug your database with the Database Inspector](#)
- [Save data in a local database using Room](#)

API references

- [androidx.room](#)
- [asLiveData\(\)](#)
- [TextView.BufferType](#)
- [AlertDialog.Builder](#)
- [ListAdapter](#)

Kotlin references

- [Extensions](#)
- [Scope functions](#)